

Datatype

September 9, 2026

1 TUPLE

Tuples are ordered, indexed, and immutable collections in Python.

```
[1]: # 1. Tuple creation
Tuple = (0, 1, 2, 3, 4, 5)

# 2. Accessing elements by index
print("Element at index 3:", Tuple[3])

# 3. Slicing: [start:stop] and [start:stop:step]
print("Slice [:3] (first 3 elements):", Tuple[:3])
print("Slice [1:6:2] (with step 2):", Tuple[1:6:2])

# 4. Arithmetic on tuple elements (tuple itself remains unchanged)
print("Tuple[5] + 5:", Tuple[5] + 5)

# 5. Membership testing using 'in' and 'not in'
print("Is 5 in Tuple?", 5 in Tuple)
print("Is 1 not in Tuple?", 1 not in Tuple)
```

```
Element at index 3: 3
Slice [:3] (first 3 elements): (0, 1, 2)
Slice [1:6:2] (with step 2): (1, 3, 5)
Tuple[5] + 5: 10
Is 5 in Tuple? True
Is 1 not in Tuple? False
```

1.0.1 Tuple Immutability

Tuples cannot be modified after creation. Attempting item assignment raises a `TypeError`.

```
[2]: # Demonstrating immutability: attempting item assignment raises TypeError
Tuple[0] = 10
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[2], line 2
      1 # Demonstrating immutability: attempting item assignment raises TypeError
```

```
----> 2 Tuple[0] = 10
```

```
TypeError: 'tuple' object does not support item assignment
```

1.0.2 Tuple Operations and Methods

```
[3]: Tuple = (0, 1, 2, 3, 4, 5)
    Tuple2 = (5, 5, 5, 6, 7)

    # Concatenation (+) creates a new combined tuple
    print("Tuple + Tuple2:", Tuple + Tuple2)

    # Built-in methods and functions
    print("Length of Tuple2:", len(Tuple2))
    print("Index of first 5:", Tuple2.index(5))
    print("Count of 5:", Tuple2.count(5))

    # Single-element tuple: must end with a comma, otherwise Python treats it as an
    # integer
    tuplele = (10,) # single element tuple should end with comma
    print("Single element tuple:", tuplele, "| Type:", type(tuplele))
```

```
Tuple + Tuple2: (0, 1, 2, 3, 4, 5, 5, 5, 5, 6, 7)
Length of Tuple2: 5
Index of first 5: 0
Count of 5: 3
Single element tuple: (10,) | Type: <class 'tuple'>
```

1.0.3 Mutable Objects Inside a Tuple

A tuple is immutable, so its element references cannot be changed. However, if an element is a mutable object (like a list), that object's contents can still be modified.

```
[4]: test = (1, [1, 2])

    # Attempting to reassign the list element raises TypeError
    test[1] = [1, 2, 3]
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[4], line 4
      1 test = (1, [1, 2])
      2
      3 # Attempting to reassign the list element raises TypeError
----> 4 test[1] = [1, 2, 3]
```

TypeError: 'tuple' object does not support item assignment

```
[5]: # Modifying the contents of the nested list is allowed
test[1][1] = [1, 2, 3]
print("Tuple after modifying inner list:", test)
```

Tuple after modifying inner list: (1, [1, [1, 2, 3]])

2 LIST

Lists are ordered, indexed, and mutable collections in Python.

```
[6]: # List creation with different datatypes
numl = [1, 2, 3, 4, 5, 6]
strl = ['a', 'b', 'c']
mixl = [1, 2, 6, 45, 'a', 'b'] # Mixed datatype list

# Identity ('is') and membership ('in')
print("numl[2] is 10:", numl[2] is 10)
print("strl[0] in mixl:", strl[0] in mixl)
```

numl[2] is 10: False
strl[0] in mixl: True

```
<>:7: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
<>:7: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
C:\Users\AV\AppData\Local\Temp\ipykernel_11724\519662203.py:7: SyntaxWarning:
"is" with 'int' literal. Did you mean "=="?
    print("numl[2] is 10:", numl[2] is 10)
```

2.0.1 Modifying Lists (Mutability)

Lists allow in-place modification of elements and slices.

```
[7]: # Modifying elements by index and slice
strl[0] = 'b'
strl[1:] = ['c', 'd']
print("After slice assignment:", strl)
```

After slice assignment: ['b', 'c', 'd']

2.0.2 List Methods: Adding, Extending, and Removing Elements

```
[8]: # 1. insert(index, item) - inserts at given index
strl.insert(0, 'a')
print("After insert(0, 'a'):", strl)

# 2. append(item) - adds item to the end
strl.append('e')
```

```

print("After append('e'):", str1)

# 3. extend(iterable) - appends all items from another list in-place
str12 = ['f', 'g', 'h', 'i', 'i']
str1.extend(str12)
print("After extend(str12):", str1)

# 4. remove(value) - removes the first occurrence of the value
str1.remove('i')
print("After remove('i'):", str1)

# 5. pop(index) - removes and returns element at index (defaults to last
    ↪ element)
popped_item = str1.pop(2)
print(f"Popped index 2 ('{popped_item}'):", str1)

popped_last = str1.pop()
print(f"Popped last element ('{popped_last}'):", str1)

```

After insert(0, 'a'): ['a', 'b', 'c', 'd']
 After append('e'): ['a', 'b', 'c', 'd', 'e']
 After extend(str12): ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'i']
 After remove('i'): ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i']
 Popped index 2 ('c'): ['a', 'b', 'd', 'e', 'f', 'g', 'h', 'i']
 Popped last element ('i'): ['a', 'b', 'd', 'e', 'f', 'g', 'h']

2.0.3 Deleting Elements (del) and Clearing a List (clear())

```

[9]: str12 = ['f', 'g', 'h', 'i', 'i']

# del removes an element at a specific index
del str12[2]
print("After del str12[2]:", str12)

# clear() removes all elements, leaving an empty list []
str12.clear()
print("After clear():", str12)

# del can also delete the variable completely from memory
del str12

```

After del str12[2]: ['f', 'g', 'i', 'i']
 After clear(): []

```

[10]: # Accessing str12 after del raises NameError because the variable was deleted
str12

```

```

NameError                                Traceback (most recent call last)
Cell In[10], line 2
      1 # Accessing strl2 after del raises NameError because the variable was_
      ↪deleted
----> 2 strl2

NameError: name 'strl2' is not defined

```

2.1 Challenge: Filter Even Numbers from List

```

[11]: noll = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Method 1: Using for loop and condition
print("Using for loop:")
for i in noll:
    if i % 2 == 0:
        print(i, end=" ")
print()

# Method 2: Using List Comprehension
even = [x for x in noll if x % 2 == 0]
print("Using list comprehension:", even)

```

Using for loop:

2 4 6 8 10

Using list comprehension: [2, 4, 6, 8, 10]

2.1.1 List Comprehension Syntax

[expression for item in iterable if condition]

2.2 Challenge: Leap Year Check from List

```

[12]: year = [2000, 1900, 2024, 2004, 2026]

# Leap year conditions:
# - Divisible by 400 -> Leap year
# - Divisible by 100 but not 400 -> Not leap year
# - Divisible by 4 but not 100 -> Leap year
for i in year:
    if i % 100 == 0:
        if i % 400 == 0:
            print(i, "is a leap year")
        else:
            print(i, "is not a leap year")
    else:
        if i % 4 == 0:

```

```
        print(i, "is a leap year")
    else:
        print(i, "is not a leap year")
```

```
2000 is a leap year
1900 is not a leap year
2024 is a leap year
2004 is a leap year
2026 is not a leap year
```